

Exam Checklist — VCE Algorithmics (HESS)

Final-week reference · exam from 26 October · bring nothing but this in your head

Command terms (what the verb demands) Proof patterns

- **State** — the fact, no working.
 - **Describe** — the what, in sentences.
 - **Explain** — the why: mechanism or reason.
 - **Justify** — evidence for a choice you made.
 - **Show / Prove** — complete logical chain; no gaps.
 - **Compare** — both sides + a conclusion.
 - **Evaluate** — judgment + criteria + verdict.
 - **Trace** — step-by-step table of state changes.
- **Induction:** base case $\rightarrow$ assume $k \rightarrow$ show $k+1 \rightarrow$ conclude.
 - **Contradiction:** assume the opposite $\rightarrow$ derive absurdity.
 - **Halting:** assume $H \rightarrow$ build D inverting $H(P, P) \rightarrow$ run $D(D) \rightarrow$ both cases contradict.
 - **Exchange (greedy/MST):** suppose optimal excludes our pick $\rightarrow$ swap in, no worse $\rightarrow$ our pick is safe.

Quotable bounds (justify on demand)

BFS/DFS	$O(V + E)$ — nodes once, edges twice
Dijkstra (heap)	$O((V + E) \log V)$
Dijkstra (array)	$O(V ^2)$ — better when dense
Prim's (heap)	$O((V + E) \log V)$
Floyd–Warshall	$\Theta(V ^3)$ — triple loop
PageRank	$O(k(V + E))$
MergeSort	$\Theta(n \log n)$ — case 2
Binary search	$\Theta(\log n)$ — halving
Naive Fibonacci	$\Theta(\phi^n)$; memoised $\Theta(n)$
Brute-force TSP	$O((n-1)!)$

Big-O working rules

- Drop constants and lower-order terms.
- Sequence adds; nesting multiplies.
- Halving/doubling loops $\rightarrow \log n$.
- Triangular loops $\rightarrow n^2$ ($\frac{1}{2}$ is a constant).
- Graphs: amortise — count total edge-touches.
- Formal def: find c, n_0 ; bound each term by the dominant one.

Master Theorem ($T(n) = aT(n/b) + f(n)$)

Compute $n^{\log_b a}$, compare with $f(n)$:

- f slower $\rightarrow \Theta(n^{\log_b a})$ (leaves win)
- f equal $\rightarrow \Theta(n^{\log_b a} \log n)$ (tie)
- f faster $\rightarrow \Theta(f(n))$ (root wins)
- Form check first! $T(n-1)$ forms $\rightarrow$ expansion.

Classes, one-liners

- P: solvable in polynomial time.
- NP: “yes” checkable in polynomial time.
- NP-complete: in NP; everything in NP reduces to it.
- Undecidable: no algorithm at all — ever.
- Reduction direction: hardness flows *to* the target.

Heuristics, the honest sentences

- Trade: optimality guarantee polynomial time.
- Hill climbing stops at *local* optima.
- Annealing: accept worse with $e^{-\Delta/T}$; cool over time.
- Always name: representation, neighbourhood, objective.

AOS3 must-knows

- Hilbert $\rightarrow$ Entscheidungsproblem $\rightarrow$ Turing 1936.
- TM: tape, head, states, transition table.
- Church–Turing: computable = TM-computable (thesis).
- Turing Test: behaviour replaces “thinking”.
- Strong vs weak AI; Chinese Room attacks *strong*.
- SVM: maximise margin; support vectors define it.
- Higher dimensions: new feature $\rightarrow$ linear separation.
- Forward prop: weighted sum + bias $\rightarrow$ activation, layer by layer. *Show the sums*.
- Overfit: train low, test high. Underfit: both high.
- Ethics: stakeholder $\rightarrow$ harm $\rightarrow$ mechanism $\rightarrow$ mitigation.

Exam craft

- Marks $\approx$ minutes: 2 marks $\approx$ 2 sentences.
- Every bound gets a justification, every claim a reason.
- Tightest bound, always; loose = lost marks.

- VCAA pseudocode: `Algorithm Name(args):`, $\leftarrow$, `Foreach/While/If ... Do, Return.`
- Attempt everything; blank pages score zero with certainty.